26CA1103E - LINUX SYSTEMS PROGRAMMING

M I N I P R O J E C T R E P O R T

TO-DO LIST MANAGEMENT SYSTEM



Team Name: [SHELLDUO]

Submitted by

S.No	Student Name	Roll No.
1	Sneha Yadav	2610520032
2	Khush Vyas	2610520010
3	Lachampet Kiran	26105200
4	Kisha Yadav	2610520043

Department	CSA – BCA
Year / Semester	I Year – I Semester
Course	26CA1103E LINUX SYSTEMS PROGRAMMING
Course Code	26CA1103E
Environment	Ubuntu Linux / WSL
Technology	Bash Shell Scripting
Course Coordinator	Dr. Alampally Sreedevi
Academic Year	2026–2027

CERTIFICATE

This is to certify that the mini project titled “To-Do List Management System” has been successfully completed by [Sneha Yadav] (R.No: [2610520032]) and [Khush] (R.No: [2610520032]) ,[Lachampet Kiran] ,[Kisha Yadav] of Team [SHELLDUO], I Year – I Semester, Department of CSA – BCA, KL University, as part of the 26CA1103E – LINUX SYSTEMS PROGRAMMING course during the academic year 2026–2027.

The project demonstrates the use of Linux command-line tools, Bash shell scripting, menudriven programming, and file handling to manage everyday tasks.

.

Dr. Alampally Sreedevi Course Coordinator	Dr. D. S. Rao HOD- CSA-BCA

DECLARATION

We hereby declare that the project report titled To-Do List Management System" is our own work, carried out as part of the 26CA1103E – LINUX SYSTEMS PROGRAMMING course in the Department of CSA – BCA, KL University. We have used Bash shell scripting and standard Linux commands to design and implement the project. The project has been prepared for academic learning and demonstration purposes.

_____ _____

[Sneha Yadav]
R.No: [2610520032]

[Khush Vyas]
R.No: [2610520010]

_____ _____

[Lachampet Kiran]
R.NO: [
Date: [13/10/2026]
Place: Hyderabad

[Kisha Yadav]
R.No: 2610520043

ACKNOWLEDGEMENT

We express our sincere gratitude to our Course Coordinator Dr. Alampally Sreedevi, Department of CSA – BCA, KL University, for the guidance and encouragement given throughout this mini project.

We also thank the Head of the Department Dr. D. S. Rao and our institution for providing the Linux / Ubuntu environment required for implementation.

This project helped us understand Linux commands, Bash scripting, variables, conditions, loops, functions, input/output, and file handling. Finally, we thank our friends and family for their support.

[Sneha Yadav] – [2610520032]
[Khush Vyas] – [2610520010]
[Lachampet Kiran] – [26105200]
[Kisha Yadav] – [2610520043]

ABSTRACT

To-Do List is a beginner-friendly Linux Bash project designed to help users manage a personal to-do list from the terminal. It provides a menu-driven interface for adding tasks, viewing saved tasks, marking tasks as completed, deleting tasks, and viewing task statistics.

The application stores task information in a text file so that tasks remain available after the program is closed and opened again. It introduces practical shell scripting concepts such as variables, read commands, loops, conditional statements, functions, case statements, and file handling.

Keywords: Linux, Bash, Shell Scripting, To-Do List, Task Management, File Handling, Terminal

TABLE OF CONTENTS

1. Introduction 1
2. Problem Statement 1
3. Objectives 1
4. Scope of the Project 1
5. Existing System 2
6. Proposed System 2
7. Requirements 2
8. System Design 3
9. Algorithm 4
10. Implementation 4
11. Source Code 4
12. Sample Execution 9
13. Testing 10
14. Team Member Responsibilities 10
15. Advantages and Limitations 10
16. Future Enhancements 11
17. Learning Outcomes 11
18. Conclusion 12
19. References 12
Appendix A – Basic Commands Used 12
Appendix B – Viva Questions 12

1. INTRODUCTION

Linux provides a powerful command-line environment in which users can automate repetitive activities using shell scripts. Bash is one of the most widely used shells in Linux. A small academic application is a useful way to understand how commands and programming logic work together.

This project uses Bash to create To-Do List, a simple menu-driven to-do list manager. Users can add tasks, view the task list, mark tasks as completed, delete tasks, and see task statistics. Task information is saved in a text file for later use.

2. PROBLEM STATEMENT

People often keep tasks in memory, on scraps of paper, or in separate notes. These methods can make it difficult to track completed work or keep a single organized list. A simple commandline application is useful for storing tasks in one place and managing their status.

The system should:

- Allow the user to add a new task.
- Display the saved task list.
- Allow a task to be marked as completed.
- Allow a task to be deleted.
- Store tasks in a file so they persist between sessions.
- Display task statistics, including total, completed, and pending tasks.

3. OBJECTIVES

- Understand the Linux terminal and basic Bash scripting.
- Use variables and user input in a shell script.
- Apply loops, conditions, and case statements to build a menu.
- Use functions to organize repeated operations.
- Read from and write to a text file.
- Practice file creation, execution, and permissions in Linux.
- Develop a small working task manager that can be demonstrated from the terminal.

4. SCOPE OF THE PROJECT

The project is intended for academic demonstration and beginner-level Linux learning. It manages a simple personal task list through the terminal and stores task data in a local text file. It does not require a database or graphical user interface.

- Platform: Ubuntu Linux / WSL.

- Interface: Menu-driven command-line terminal.
- Input: Task descriptions and menu choices.
- Output: Task list, completion status, and task statistics.
- Users: Students or anyone who wants a basic terminal-based task list.

5. EXISTING SYSTEM

In a manual system, tasks may be written on paper or kept in separate notes. It can be difficult to update the list, identify completed tasks, and maintain one consistent record.

Limitations:

- Tasks can be forgotten when recorded in different places.
- Manually updating lists can take additional time.
- Completed and pending tasks may not be easy to distinguish.
- A paper-based list does not demonstrate Linux automation or file handling.

6. PROPOSED SYSTEM

The proposed system uses a Bash shell script to provide a menu for managing tasks. Users choose an operation, enter the required information, and receive a message describing the result. Task data is stored in a local text file.

Basic workflow:

1. Start the To-Do List Bash script.
2. Display the main menu.
3. Choose an operation such as Add, View, Complete, Delete, or Statistics.
4. Read the task description or task number when required.
5. Update the task file according to the selected operation.
6. Check user input and handle invalid menu choices.
7. Display the updated task list or statistics.
8. Return to the main menu.
9. Exit when the user chooses the exit option.

7. REQUIREMENTS

7.1 Hardware Requirements

- Computer or laptop.
- Minimum 4 GB RAM recommended.
- Keyboard and display.

7.2 Software Requirements

- Ubuntu Linux or Ubuntu through Windows Subsystem for Linux (WSL).
- Bash shell.
- Terminal application.
- Any text editor such as nano or vim.

8. SYSTEM DESIGN

8.1 Input

- Menu choice selected by the user.
- Task description for a new task.
- Task number to mark as completed.
- Task number to delete.
- No additional input for viewing tasks.
- No additional input for task statistics.

8.2 Processing

- Read the existing task file, if available.
- Add a new task or display the saved tasks.
- Update the selected task's completion status or remove it.
- Calculate total, completed, and pending task counts.

8.3 Output

- A numbered list of tasks.
- A confirmation message after an operation.
- Completion status for each task.
- Total number of tasks.
- Number of completed and pending tasks.

8.4 Task Status

Task Status	Symbol / Value	Meaning
Pending	[ ]	Task is not yet completed
Completed	[✓]	Task has been completed
Total Tasks	Number	All tasks saved in the list
Completed Tasks	Number	Tasks marked as completed

Pending Tasks	Number	Tasks still to be completed
Data File	tasks.txt	Stores task descriptions and status
Interface	Menu	User selects an operation

9. ALGORITHM

1. Start the program.
2. Create or open the task data file.
3. Display the menu of available operations.
4. Read the user's menu choice.
5. If Add is selected, read the task description and save it to the file.
6. If View is selected, display the saved tasks with their status.
7. If Complete is selected, update the chosen task as completed.
8. If Delete is selected, remove the chosen task from the file.
9. If Statistics is selected, count total, completed, and pending tasks.
10. Repeat the menu until the user chooses Exit.

10. IMPLEMENTATION

10.1 Create the Project Directory

```
mkdir TO_DO_LIST cd
TO_DO_LIST
```

10.2 Create the Bash File

```
nano todo.sh
```

10.3 Give Execute Permission

```
chmod +x todo.sh
```

10.4 Run the Project

```
./todo.sh
```

11. SOURCE CODE

!/bin/bash

TASK_FILE="$HOME/To-Do/tasks.txt"

if [! -f "$TASK_FILE "]; then

mkdir -p "$HOME/To-Do"

```
touch "$TASK_FILE"
fi
while true
do
 echo ""
   echo "========================================"
   echo "          WELCOME TO To-Do"
echo ""
   echo "       Your Personal Task Manager"
   echo "========================================"
   echo ""

   echo "========================================"
   echo "               MY TASKS"
   echo "========================================"
   echo ""

   if [ -s "$TASK_FILE" ]; then
      cat -n "$TASK_FILE"
   else
      echo "  No tasks yet. Add your first task!"
   fi

   echo ""
   echo "========================================"
   echo ""

   echo "      WHAT WOULD YOU LIKE TO DO?"
   echo ""
   echo "1. Add a New Task"
   echo ""
   echo "2. Search for a Task"
   echo ""
```

```
    echo "3. Mark Task as Completed"
    echo ""
    echo "4. Delete a Task"
    echo ""
    echo "5. View Task Statistics"
    echo ""
    echo "6. Exit"
    echo ""

    read -p "Enter your choice: " choice

    case "$choice" in
        1)
read -p "Enter your task: " task
if [ -n "$task" ]; then
echo "[ ] $task" >> "$TASK_FILE"
echo "Task added succesfully!"
else
echo "Task cannot be empty."
fi
;;
2)    read -p "Enter a word to search for: " keyword

    echo ""
    echo "========================================"
    echo "           SEARCH RESULTS"
    echo "========================================"
    echo ""

    if [ -z "$keyword" ]; then
        echo "Please enter a search word."
    elif grep -inF -- "$keyword" "$TASK_FILE"; then
        echo ""
```

```
    echo "Search completed."
  else
    echo "No matching tasks found."
  fi

  echo ""
  echo "========================================"
  read -p "Press Enter to continue..."
  ;;
3)   if [ ! -s "$TASK_FILE" ]; then
    echo "No tasks found."
  else
    cat -n "$TASK_FILE"
    read -p "Enter task number to complete: " num
    total=$(wc -l < "$TASK_FILE")
    if [[ "$num" =~ ^[0-9]+$ ]] && [ "$num" -ge 1 ] && [ "$num" -le "$total" ]; then
      sed -i "${num}s/^\[ \]/[✓]/" "$TASK_FILE"
      echo "Task marked as completed!"
    else
      echo "Invalid task number."
    fi
  fi
  ;;

    4)   if [ ! -s "$TASK_FILE" ]; then
    echo "No tasks found."
  else
    cat -n "$TASK_FILE"
    read -p "Enter task number to delete: " num
    total=$(wc -l < "$TASK_FILE")
    if [[ "$num" =~ ^[0-9]+$ ]] && [ "$num" -ge 1 ] && [ "$num" -le "$total" ]; then
      sed -i "${num}d" "$TASK_FILE"
      echo "Task deleted successfully!"
```

```
        else
            echo "Invalid task number."
        fi
    fi
    ;;
5}  total=$(wc -l < "$TASK_FILE")
    completed=$(grep -c '^\[✓\]' "$TASK_FILE")
    pending=$(grep -c '^\[ \]' "$TASK_FILE")
    echo "===== TASK STATISTICS ====="
    echo "Total tasks: $total"
    echo "Completed tasks: $completed"
    echo "Pending tasks: $pending"
    ;;
        6) echo "Thank you for using To-Do list. Have a productive day!"; break ;;
        *) echo "Invalid choice. Try again." ;;
    esac
done
```

12. SAMPLE EXECUTION

12.1 Successful Execution

```
sneha27@DESKTOP-72TEHRP:~/TO_DO_LIST$ bash todo.sh

========================================
          WELCOME TO To-Do

        Your Personal Task Manager
========================================

========================================
                MY TASKS
========================================

  No tasks yet. Add your first task!

========================================

       WHAT WOULD YOU LIKE TO DO?

1. Add a New Task

2. Search for a Task

3. Mark Task as Completed

4. Delete a Task

5. View Task Statistics

6. Exit

Enter your choice: 1
Enter your task: linux project
Task added succesfully!

========================================
          WELCOME TO To-Do

        Your Personal Task Manager
========================================

========================================
                MY TASKS
========================================

     1  [ ] linux project

========================================

       WHAT WOULD YOU LIKE TO DO?

1. Add a New Task

2. Search for a Task

3. Mark Task as Completed

4. Delete a Task

5. View Task Statistics

6. Exit

Enter your choice:
```

12.2 Invalid Input Example

```
Invalid menu choice example: 9
Invalid choice. Please select 1 to
6.
The program returns to the menu after an invalid choice.
```

13. TESTING

The following test cases cover task creation, viewing, completion, deletion, file persistence, statistics, and invalid menu choices. Run each case against the final script and update the status according to the observed result.

Test	Action	Expected Output	Expected Status	Input / Condition	Result
1	Add a task	Task is saved	To verify	Enter a task description	Pending
2	Search for a task	Saved tasks are listed	To verify	Task exists in file	Displayed
3	Mark completed	Task status changes	To verify	Choose a valid task number	Completed
4	Delete task	Selected task is removed	To verify	Choose a valid task number	Deleted
5	Restart program	Saved tasks remain	To verify	Check file persistence	Retained
6	View statistics	Counts are displayed	To verify	Total = completed + pending	Displayed

TEAM MEMBER RESPONSIBILITIES

Team Member	Responsibilities	Deliverables
[SNEHA YADAV]	Project logic, Bash functions, task operations, file handling	Working source code and algorithm
[KHUSH VYAS]	Testing, sample outputs, documentation, presentation	Test cases, project report, and PPT

14. ADVANTAGES AND LIMITATIONS

14.1 Advantages

- Simple, menu-driven interface for beginners.
- Runs directly in the Linux terminal.
- Keeps everyday tasks organized in one list.
- Allows tasks to be marked as completed.

- Stores task information in a text file without requiring a database.
- Can be extended with search, priorities, and due dates.

14.2 Limitations

- Requires access to a Linux terminal or compatible Bash environment.
- Uses a local text file rather than a database.
- Task descriptions are stored as plain text.
- The current version is intended for a single user on one computer.

15. FUTURE ENHANCEMENTS

- Add task priorities such as High, Medium, and Low.
- Add due dates and reminders.
- Add a search feature for finding tasks.
- Export the task list to a separate report file.
- Add task categories or filters for pending and completed tasks.
- Improve input validation and file backup.
- Add optional sorting by date or priority.

16. LEARNING OUTCOMES

After completing this project, students should be able to:

- Navigate directories using Linux commands.
- Create and edit files from the terminal.
- Change file permissions using chmod.
- Write and execute a Bash script.
- Use variables and read for input.
- Use loops, conditions, and case statements.
- Organize code using functions.
- Read from and write to a text file.
- Validate menu choices and task numbers.
- Test and debug a basic Linux shell program.

17. CONCLUSION

To-Do List demonstrates how Linux and Bash shell scripting can be used to solve a simple everyday problem. The menu-driven program allows users to add, view, complete, and delete tasks, while a text file provides basic persistence between sessions.

As a beginner-level project, To-Do List provides practical experience with Linux commands, file permissions, variables, loops, conditional statements, functions, and file handling. It can be extended in the future with priorities, due dates, search, and other task-management features.

18. REFERENCES

- The Linux man-pages: bash, chmod, mkdir, cd, nano, cat, and related commands.
- GNU Bash Reference Manual.
- Ubuntu Linux documentation.
- Course notes and laboratory material provided by the faculty.

APPENDIX A – BASIC COMMANDS USED

Command	Purpose
pwd	Display current directory
ls	List files and directories
cd TO_DO_LIST	Open the project directory
mkdir TO_DO_LIST	Create the project directory
nano todo.sh	Create or edit the Bash script
chmod +x todo.sh	Give execute permission
./todo.sh	Run the To-Do List program
cat tasks.txt	View saved task data

APPENDIX B – VIVA QUESTIONS

No.	Question
1	What is Linux?
2	What is Bash?
3	What is a shell script?
4	Why is chmod +x used?
5	What is the purpose of the read command?

6	Why is a case statement useful in a menu?
7	How does To-Do List store task information?
8	How can a task be marked as completed?
9	What is file handling in Bash?
10	How could To-Do List be improved in the future?